

```
// - перезаписываем существующие элементы в диапазон назначения
copy (coll1.cbegin(), coll1.cend(), // источник
      coll2.begin());              // назначение

// создаем третью коллекцию, имеющую достаточный объем
// - начальный размер передается как параметр
deque<int> coll3(coll1.size());

// копируем элементы из первой в третью коллекцию
copy (coll1.cbegin(), coll1.cend(), // источник
      coll3.begin());              // назначение
}
```

Здесь функция `resize()` используется для изменения количества элементов в существующем контейнере `coll2`.

```
coll2.resize (coll1.size());
```

Затем коллекция `coll3` инициализируется специальным начальным размером, обеспечивающим достаточный объем для хранения всех элементов коллекции `coll1`.

```
deque<int> coll3(coll1.size());
```

Отметим, что изменение и инициализация размера приводят к созданию новых элементов, которые инициализируются своим конструктором, заданным по умолчанию, поскольку им не передаются никакие аргументы. Кроме того, конструктору и функции `resize()` можно передать дополнительный аргумент для инициализации новых элементов.

6.5. Адаптеры итераторов

Итераторы являются *чистой абстракцией*: все, что *ведет себя* как итератор, является итератором. По этой причине можно написать классы, имеющие интерфейс итераторов, но делающие нечто совершенно другое. Стандартная библиотека C++ содержит несколько стандартных итераторов особого вида: *адаптеры итераторов*. Это нечто большее, чем просто вспомогательные классы; они придают концепции итераторов много больше мощи.

В следующих подразделах рассматриваются перечисленные ниже адаптеры итераторов.

1. Итераторы вставки.
2. Поточковые итераторы.
3. Обратные итераторы.
4. Итераторы перемещения (начиная со стандарта C++11).

Подробное описание адаптеров итераторов приведено в разделе 9.4.

6.5.1. Итераторы вставки

Итераторы вставки используются для того, чтобы дать алгоритмам возможность работать в режиме вставки, а не замены. В частности, итераторы вставки решают проблему, возникающую, когда в диапазоне назначения нет достаточного места: они позволяют увеличивать размер диапазона вставки.

Итераторы вставки переопределяют свой интерфейс следующим образом.

- Если вы присваиваете значение элементу итератора вставки, он вставляет это значение в коллекцию, которой принадлежат итератор. Три разных итератора вставки имеют разные возможности для вставки элементов — в начало, в конец и в заданную позицию.
- Вызов для перемещения вперед является пустой операцией.

Итераторы с таким интерфейсом относятся к категории итераторов вывода, способных записывать и присваивать значения только при перемещении вперед (см. раздел 9.2).

Рассмотрим пример:

```
// stl/copy2.cpp

#include <algorithm>
#include <iterator>
#include <list>
#include <vector>
#include <deque>
#include <set>
#include <iostream>
using namespace std;

int main()
{
    list<int> coll1 = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
    // копируем элементы коллекции coll1 в коллекцию coll2,
    // добавляя их к существующим

    vector<int> coll2;
    copy (coll1.cbegin(), coll1.cend(), // источник
          back_inserter(coll2));        // назначение

    // копируем элементы из коллекции coll1 в коллекцию coll3,
    // вставляя их в начало
    // - изменяем порядок на обратный
    deque<int> coll3;
    copy (coll1.cbegin(), coll1.cend(), // источник
          front_inserter(coll3));       // назначение

    // копируем элементы коллекции coll1 в коллекцию coll4
    // - итератор, работающий только с ассоциативными коллекциями
    set<int> coll4;
    copy (coll1.cbegin(), coll1.cend(), // источник
          inserter(coll4, coll4.begin())); // назначение
}
```

Этот пример демонстрирует все три стандартных итератора вставки.

1. **Итераторы вставки в конец** вставляют элементы в конец своего контейнера (добавляет их), вызывая функцию `push_back()`. Например, в следующем примере все элементы коллекции `coll1` добавляются в коллекцию `coll2`:

```
copy (coll1.cbegin(), coll1.cend(), // источник
      back_inserter(coll2));      // назначение
```

2. Разумеется, итераторы вставки в конец можно использовать только для контейнеров, имеющих функцию-член `push_back()`. В стандартной библиотеке C++ к таким контейнерам относятся `vector`, `deque`, `list` и строки.
3. **Итераторы вставки в начало** вставляют элементы в начало своего контейнера, вызывая функцию `push_front()`. Например, следующий оператор вставляет все элементы коллекции `coll1` в коллекцию `coll3`:

```
copy (coll1.cbegin(), coll1.cend(), // источник
      front_inserter(coll3));      // назначение
```

4. Отметим, что этот вид итераторов вставки изменяет порядок вставленных элементов на противоположный. Если вставить в начало 1, а затем 2, то 1 будет следовать за 2.
5. Итераторы вставки в начало можно использовать только в контейнерах, имеющих функцию-член `push_front()`. В стандартной библиотеке C++ к таким контейнерам относятся `deque`, `list` и `forward_list`.
6. **Обобщенные итераторы вставки** вставляют элементы непосредственно перед позицией, передаваемой как второй аргумент во время инициализации. Обобщенный итератор вставки вызывает функцию-член `insert()` с новым значением и новой позицией, задаваемыми аргументами. Отметим, что функцию-член `insert()` содержат все стандартные контейнеры, за исключением `array` и `forward_list`. Таким образом, этот итератор является единственным итератором для работы со стандартными ассоциативными и неупорядоченными контейнерами.
7. Однако постойте! Передавать позицию для вставки элемента в ассоциативный или неупорядоченный массив не слишком полезно, не так ли? В ассоциативных контейнерах позиции зависят от значений *элементов*, а в неупорядоченных контейнерах позиция элемента не определена. Решение простое: для ассоциативных и неупорядоченных контейнеров передаваемая позиция считается *подсказкой* для начала поиска правильной позиции. Однако контейнер может ее игнорировать. В разделе 9.6 описываются итераторы вставки, определенные пользователем, которые могут оказаться более полезными для работы с ассоциативными и неупорядоченными контейнерами.

Функциональные возможности итераторов вставки перечислены в табл. 6.1. Дополнительные подробности приведены в разделе 9.4.2.

Таблица 6.1. Стандартные итераторы вставки

Выражение	Вид итератора вставки
<code>back_inserter(контейнер)</code>	Добавляет элементы в том же порядке, используя функцию <code>push_back(значение)</code>
<code>front_inserter(контейнер)</code>	Вставляет элемент в начало в обратном порядке, используя функцию <code>push_front(значение)</code>
<code>inserter(контейнер, позиция)</code>	Вставляет элементы в указанную <i>позицию</i> (в том же порядке), используя функцию <code>insert(позиция, значение)</code>

6.5.2. Потокковые итераторы

Потокковые итераторы считывают данные из потока или записывают данные в поток⁹. Таким образом, они создают абстракцию, позволяющую вводить данные с клавиатуры, которая рассматривается как коллекция, предназначенная для чтения. Аналогично с помощью потокового итератора можно перенаправить вывод алгоритма непосредственно в файл или на экран.

Следующий пример демонстрирует мощь всей библиотеки STL. По сравнению с обычными программами на языке C или C++, этот пример выполняет очень сложную работу с помощью нескольких операций.

```
// stl/ioiter1.cpp

#include <iterator>
#include <algorithm>
#include <vector>
#include <string>
#include <iostream>
using namespace std;

int main()
{
    vector<string> coll;
    // считываем все слова из стандартного потока ввода
    // - источник: все строки вплоть до конца файла (или ошибки)
    // - назначение: coll (вставка)
    copy (istream_iterator<string>(cin), // начало источника
          istream_iterator<string>(),    // конец источника
          back_inserter(coll));          // назначение

    // сортируем элементы
    sort (coll.begin(), coll.end());

    // выводим все элементы без дубликатов
    // - источник: coll
    // - назначение: стандартный поток вывода
    // (с символом перехода на новую строку между элементами)
    unique_copy (coll.cbegin(), coll.cend(), // источник
                 ostream_iterator<string>(cout, "\n")); // назначение
}
```

Эта программа содержит только три оператора, которые считывают все слова из стандартного потока ввода и выводят их упорядоченный список. Рассмотрим эти три оператора по очереди. В операторе

```
copy (istream_iterator<string>(cin),
      istream_iterator<string>(),
      back_inserter(coll));
```

используются два потоковых итератора ввода.

⁹ Поток — это объект, представляющий каналы ввода–вывода (см. главу 15).

1. Выражение

```
istream_iterator<string>(cin)
```

2. создает потоковый итератор, считывающий данные из стандартного потока ввода `cin`. Шаблонный аргумент `string` указывает, что потоковый итератор считывает элементы данного типа (строки описаны в главе 13). Эти элементы считываются с помощью обычной операции ввода `>>`. Таким образом, каждый раз, когда алгоритм хочет обработать новый элемент, потоковый итератор ввода трансформирует это желание в вызов `cin >> строка`.
3. Операция ввода строк обычно считывает одно слово, отделенное разделителями (см. раздел 13.2.10), так что алгоритм считывает слово за словом.

4. Выражение

```
istream_iterator<string>()
```

5. вызывает конструктор потоковых итераторов, заданный по умолчанию, который создает так называемый *итератор конца потока* (end-of-stream iterator). Он представляет поток, чтение из которого больше невозможно.

Как обычно, алгоритм `copy()` работает, только если первый аргумент (инкрементированный) отличается от второго. Итератор конца потока используется как *конец диапазона* (end of the range), поэтому алгоритм считывает все строки из потока `cin`, пока может (т.е. пока не достигнет конца потока или не произойдет ошибка). Подводя итоги, отметим, что источником алгоритма являются “все слова, считанные из потока `cin`”. Эти слова копируются путем вставки в коллекцию `coll` с помощью обратного итератора.

Алгоритм `sort()` сортирует все элементы:

```
sort (coll.begin(), coll.end());
```

И наконец, оператор

```
unique_copy (coll.cbegin(), coll.cend(),  
ostream_iterator<string>(cout, "\n"));
```

копирует все элементы из коллекции в выходной поток `cout`. На протяжении этого процесса алгоритм `unique_copy()` удаляет дубликаты. Выражение

```
ostream_iterator<string>(cout, "\n")
```

создает итератор потока вывода, записывающий объекты класса `string` в поток `cout`, выполняя операцию `<<` для каждого элемента.

Второй аргумент, следующий за `cout`, является необязательным и служит в качестве разделителя между элементами. В нашем примере этим разделителем является символ перехода на новую строку, поэтому каждый элемент будет записан в отдельной строке.

Все компоненты нашей программы являются шаблонными, поэтому ее легко настроить на сортировку любых других типов, таких как целые числа или более сложные объекты. Более подробное описание и примеры использования итераторов ввода-вывода приведены в разделе 9.4.3.

В нашем примере для сортировки всех слов, введенных из стандартного потока ввода, использовалось одно объявление и три инструкции. Однако то же самое можно сделать

с помощью одного объявления и одной инструкции. Соответствующий пример приведен в разделе 1.

Начиная со стандарта C++11 в качестве параметра, задающего конца диапазона можно передавать пустые фигурные скобки, а не итератор потока, созданный по умолчанию. Это возможно благодаря тому, что тип аргумента, определяющего конец диапазона, выводится из предыдущего аргумента, определяющего начало диапазона:

```
copy (istream_iterator<string>(cin), // начало диапазона-источника
      {}, // конец диапазона-источника
      back_inserter(coll)); // диапазон-получатель
```

6.5.3. Обратные итераторы

Обратные итераторы позволяют алгоритмам перебирать элементы контейнера в обратном направлении, заменяя операцию инкремента на операцию декремента, и наоборот. Все контейнеры с двунаправленными итераторами или операциями произвольного доступа (т.е. все последовательные контейнеры, за исключением `forward_list` и всех ассоциативных контейнеров) могут создавать обратные итераторы с помощью функций-членов `rbegin()` и `rend()`. В соответствии со стандартом C++11 в контейнерах также предусмотрены функции-члены, возвращающие итераторы, предназначенные только для чтения, `crbegin()` и `crend()`.

В контейнерах `forward_list` и неупорядоченных контейнерах интерфейс для обратного обхода (`rbegin()`, `rend()` и т.д.) не предусмотрен. Причина заключается в том, что их реализация для обхода элементов использует только односвязные списки.

Рассмотрим следующий пример:

```
// stl/reviter1.cpp

#include <iterator>
#include <algorithm>
#include <vector>
#include <iostream>
using namespace std;

int main()
{
    vector<int> coll;

    // вставляем элементы от 1 до 9
    for (int i=1; i<=9; ++i) {
        coll.push_back(i);
    }

    // выводим на экран все элементы в обратном порядке
    copy (coll.crbegin(), coll.crend(), // источник
           ostream_iterator<int>(cout, " ")); // назначение
    cout << endl;
}
```

Следующее выражение возвращает обратный итератор, предназначенный только для чтения элементов контейнера `coll`.

```
coll.crbegin()
```

Этот итератор можно использовать в качестве начала обратного обхода элементов коллекции. Он установлен на последний элемент коллекции. Таким образом, следующее выражение возвращает значение последнего элемента:

```
*coll.crbegin()
```

Соответственно, следующее выражение возвращает обратный итератор для коллекции `coll`, который можно использовать в качестве конца для обратного обхода:

```
coll.crend()
```

Как обычно при работе с диапазонами, позиция итератора следует за последним элементом, но в обратном направлении; иначе говоря, она расположена *перед* первым элементом коллекции.

Никогда не следует применять операцию `*` (или операцию `->`) к позиции, не содержащей корректного элемента. Таким образом, выражение

```
*coll.crend()
```

не определено, так же как и выражения `*coll.end()` и `*coll.cend()`.

Преимущество использования обратных итераторов заключается в том, что все алгоритмы способны обходить элементы коллекции в обратном направлении без специального кода. Переход к следующему элементу с помощью операции `++` переопределяется с помощью операции `--`. Например, в нашем случае алгоритм `copy()` обходит элементы коллекции `coll` от последнего к первому элементу. Таким образом, получаем следующие результаты работы программы:

```
9 8 7 6 5 4 3 2 1
```

Обычные итераторы также можно превращать в обратные, и наоборот. Однако размыванное значение итератора при этом изменяется. Подробности, касающиеся обратных итераторов, изложены в разделе 9.4.1.

6.5.4. Итераторы перемещения

Итераторы перемещения введены в стандарте C++11. Они превращают любой доступ к элементу в операцию перемещения. Это позволяет перемещать элементы из одного контейнера в другой либо в конструкторах, либо с помощью алгоритмов. Подробности изложены в разделе 9.4.4.

6.6. Пользовательские обобщенные функции

Библиотека STL допускает расширение. Это означает, что пользователь может писать свои функции и алгоритмы для работы с элементами коллекции. Разумеется, эти операции могут быть обобщенными. Однако для объявления корректного итератора в этих операциях необходимо использовать тип контейнера, который у каждого контейнера свой. Для того чтобы облегчить создание обобщенных функций, каждый контейнерный тип предусматривает внутренние определения типов. Рассмотрим следующим пример:

```
// stl/print.hpp

#include <iostream>
#include <string>

// PRINT_ELEMENTS()
// - выводит необязательную строку optstr, за которой следуют
// - все элементы коллекции coll
// - в одной строке, разделенные пробелами
template <typename T>
inline void PRINT_ELEMENTS (const T& coll,
                           const std::string& optstr="")
{
    std::cout << optstr;
    for (const auto& elem : coll) {
        std::cout << elem << ' ';
    }
    std::cout << std::endl;
}
```

В этом примере определена обобщенная функция, которая выводит на экран необязательную строку, за которой следуют все элементы переданного контейнера.

До появления стандарта C++11 цикл по элементам выглядел следующим образом:

```
typename T::const_iterator pos;
for (pos=coll.begin(); pos!=coll.end(); ++pos) {
    std::cout << *pos << ' ';
}
```

где переменная `pos` объявлена как итератор передаваемого контейнера. Отметим, что ключевое слово `typename` здесь является обязательным, чтобы указать, что `const_iterator` — это тип, а не статический член типа `T` (см. раздел 3.2).

В дополнение к типам `iterator` и `const_iterator` контейнеры содержат другие типы для создания обобщенных функций. Например, они предоставляют тип элементов для создания временных копий элементов. Подробности изложены в разделе 9.5.1.

Необязательный второй аргумент `PRINT_ELEMENTS` представляет собой строку, используемую как префикс перед записываемыми элементами. Таким образом, функция `PRINT_ELEMENTS()` позволяет комментировать результаты работы программы:

```
PRINT_ELEMENTS (coll, "все элементы: ");
```

Эта функция описана здесь потому, что она будет широко использоваться в остальной части книги при выводе элементов контейнера.

6.7. Модифицирующие алгоритмы

До сих пор мы рассматривали концепцию библиотеки STL в целом. Контейнеры представляют разные способы управления коллекциями данных, алгоритмы выполняют операции чтения и записи элементов этих коллекций. Итераторы служат посредниками между контейнерами и алгоритмами. Итераторы, предоставляемые контейнерами, позволяют перебирать все элементы в разном порядке и в разных режимах, например в режиме вставки.

Однако настало время сказать “однако”. На практике работа с библиотекой STL связана с определенными ограничениями, которые следует знать. Многие из них относятся к модификациям.

Несколько алгоритмов модифицируют целевые диапазоны. В частности, эти алгоритмы могут удалять элементы. При этом возникают определенные аспекты, о которых речь пойдет в этом разделе. Эти аспекты вызывают удивление и демонстрируют обратную сторону концепции STL, которая подразумевает разделение контейнеров и алгоритмов для достижения высокой гибкости.

6.7.1. Удаление элементов

Алгоритм `remove()` удаляет элементы из диапазона. Однако применение этого алгоритма ко всем элементам контейнера приводит к удивительным результатам. Рассмотрим следующий пример:

```
// stl/remove1.cpp

#include <algorithm>
#include <iterator>
#include <list>
#include <iostream>
using namespace std;

int main()
{
    list<int> coll;

    // вставляем элементы от 6 до 1 и от 1 до 6
    for (int i=1; i<=6; ++i) {
        coll.push_front(i);
        coll.push_back(i);
    }

    // выводим на экран все элементы коллекции
    cout << "pre: ";
    copy (coll.cbegin(), coll.cend(),           // источник
          ostream_iterator<int>(cout, " ")); // назначение
    cout << endl;

    // удаляем все элементы, равные 3
    remove (coll.begin(), coll.end(), // диапазон
            3;                       // значение

    // выводим на экран все элементы коллекции
    cout << "post: ";
    copy (coll.cbegin(), coll.cend(),           // источник
          ostream_iterator<int>(cout, " ")); // назначение
    cout << endl;
}
```

Человек, поверхностно разбирающийся в библиотеке STL, при чтении этой программы мог бы ожидать, что из коллекции будут удалены все элементы, равные 3. Однако результат программы выглядит следующим образом:

```
pre: 6 5 4 3 2 1 1 2 3 4 5 6
post: 6 5 4 2 1 1 2 4 5 6 5 6
```

Таким образом, алгоритм `remove()` не изменяет количество элементов в коллекции, к которой он применяется. Функция-член `cend()` возвращает старый конец — как и функция `end()`, — а функция `size()` возвращает старое количество элементов. Однако кое-что изменилось: элементы изменили порядок следования, как если бы они были удалены. Каждый элемент, равный 3, заменяется следующими элементами (см. рис. 6.10). В конце коллекции старые элементы не заменяются алгоритмом и остаются неизменными. Логически эти элементы больше не принадлежат коллекции.

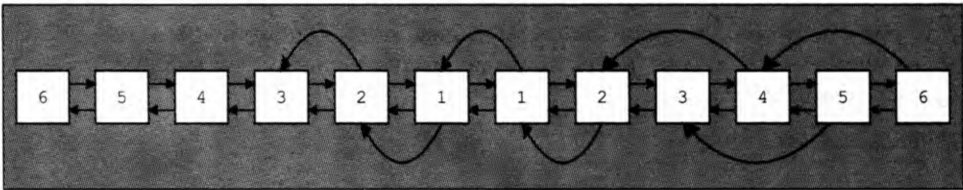


Рис. 6.10. Как работает алгоритм `remove()`

Однако этот алгоритм возвращает новый логический конец. С помощью этого алгоритма можно получить доступ к результирующему диапазону, уменьшить размер коллекции или подсчитать количество удаленных элементов. Рассмотрим следующую модифицированную версию этого примера:

```
// stl/remove2.cpp

#include <algorithm>
#include <iterator>
#include <list>
#include <iostream>
using namespace std;

int main()
{
    list<int> coll;

    // вставляем элементы от 6 до 1 и от 1 до 6
    for (int i=1; i<=6; ++i) {
        coll.push_front(i);
        coll.push_back(i);
    }

    // выводим на экран все элементы коллекции
    copy (coll.cbegin(), coll.cend(),
          ostream_iterator<int>(cout, " "));
    cout << endl;

    // удаляем все элементы, равные 3
    // - сохраняем новый конец
    list<int>::iterator end = remove (coll.begin(), coll.end(),
                                     3);
```

```

// выводим на экран результирующие элементы коллекции
copy (coll.begin(), end,
      ostream_iterator<int>(cout, " "));
cout << endl;

// выводим на экран количество удаленных элементов
cout << "number of removed elements "
      << distance(end, coll.end()) << endl;

// удаляем "удаленные" элементы
coll.erase (end, coll.end());

// выводим на экран все элементы модифицированной коллекции
copy (coll.cbegin(), coll.cend(),
      ostream_iterator<int>(cout, " "));
cout << endl;
}

```

В этой версии значение, возвращаемое алгоритмом `remove()`, присваивается итератору `end`:

```
list<int>::iterator end = remove (coll.begin(), coll.end(), 3);
```

Это новый логический конец модифицированной коллекции после “удаления” элементов. Это возвращаемое значение можно использовать в качестве нового конца для последующих операций:

```
copy (coll.begin(), end, ostream_iterator<int>(cout, " "));
```

Отметим, что мы должны использовать функцию `begin()`, а не `cbegin()`, потому что итератор `end` определен как неконстантный, а начало и конец диапазона определены как итераторы одинакового типа.

Другая возможность — вычислить количество удаленных элементов, определив расстояние между “логическим” и реальным концом коллекции.

```
cout << "number of removed elements: "
<< distance(end, coll.end()) << endl;
```

Здесь используется специальная вспомогательная функция `distance()` для итераторов, которая возвращает расстояние между итераторами. Если бы итераторы были итераторами произвольного доступа, можно было бы вычислить разность непосредственно с помощью операции `-`. Однако в данном случае контейнер является списком, поэтому он предусматривает двунаправленные итераторы. Подробное описание функции `distance()` приведено в разделе 9.3.3.

Если действительно необходимо удалить “удаленные” элементы, тогда нужно вызвать соответствующую функцию-член контейнера. Для этой цели контейнеры содержат функцию-член `erase()`, удаляющую весь диапазон, заданный с помощью аргументов.

```
coll.erase (end, coll.end());
```

Результаты работы всей программы выглядят следующим образом:

```
6 5 4 3 2 1 1 2 3 4 5 6
6 5 4 2 1 1 2 4 5 6
number of removed elements: 2
6 5 4 2 1 1 2 4 5 6
```

Если необходимо удалить элементы с помощью одного оператора, можно выполнить инструкцию

```
coll.erase (remove(coll.begin(), coll.end(), 3), coll.end());
```

Почему алгоритмы сами не вызывают функцию `erase()`? Этот вопрос подчеркивает цену, которую приходится платить за гибкость библиотеки STL. Библиотека STL отделяет структуры данных от алгоритмов, используя итераторы в качестве интерфейса. Однако итераторы — это абстракция позиции в контейнере. В принципе, итераторы *ничего не знают* о своих контейнерах. Таким образом, алгоритмы, использующие итераторы для доступа к элементам контейнера, не могут вызывать его функции-члены.

Эта схема имеет важные последствия, поскольку позволяет алгоритмам работать с диапазоном, которые отличаются от “всех элементов контейнера”. Например, диапазон может быть подмножеством всех элементов коллекции. Он даже может быть контейнером, не имеющим функции-члена `erase()` (примером такого контейнера является массив). Итак, для того чтобы алгоритм был как можно более гибким, желательно не требовать, чтобы итератор знал о контейнере.

Отметим, что физически удалять “удаленные” элементы часто не требуется. Во многих ситуациях нет ничего страшного в том, что вместо реального конца контейнера возвращается его логический конец. В частности, все алгоритмы можно применять, указывая этот новый логический конец.

6.7.2. Работа с ассоциативными и неупорядоченными контейнерами

Модифицирующие алгоритмы, т.е. алгоритмы, удаляющие элементы, изменяющие их порядок или их значения, порождают другую проблему при их использовании для ассоциативных или неупорядоченных контейнеров: такие контейнеры нельзя использовать в качестве целевого диапазона. Причина проста: при работе с ассоциативными и неупорядоченными контейнерами модифицирующие алгоритмы могут изменять значения или позиции элементов, тем самым нарушая порядок следования элементов в контейнере (упорядоченный в ассоциативных контейнерах или определенный хеш-функцией в неупорядоченных контейнерах). Для того чтобы избежать нарушения внутреннего порядка, каждый итератор для ассоциативного и неупорядоченного контейнера объявляется как итератор с константным значением или ключом. В результате манипуляция элементами в ассоциативных или неупорядоченных контейнерах распознается на этапе компиляции как ошибка¹⁰.

Из-за этой проблемы также нельзя выполнять алгоритмы удаления для ассоциативных контейнеров, потому что эти алгоритмы неявно манипулируют элементами. Значения удаленных элементов перезаписываются следующими элементами, которые не удаляются.

¹⁰К сожалению, некоторые системы плохо обрабатывают ошибки. Вы можете видеть, что нечто происходит неправильно, но не понимать почему.

Возникает вопрос: как же удалить элементы из ассоциативных контейнеров? Ответ простой: вызвать их функции-члены! Каждый ассоциативный и неупорядоченный контейнер содержит функцию-член для удаления элементов. Например, для удаления элементов можно вызвать функцию-член `erase()`.

```
// stl/remove3.cpp

#include <set>
#include <algorithm>
#include <iterator>
#include <iostream>
using namespace std;

int main()
{
    // неупорядоченное множество с элементами от 1 до 9
    set<int> coll = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };

    // выводим на экран все элементы коллекции
    copy (coll.cbegin(), coll.cend(),
          ostream_iterator<int>(cout, " "));
    cout << endl;

    // Удаляем все элементы, значение которых равно 3,
    // - алгоритм remove() не работает
    // - вместо него применяется функция-член erase()
    int num = coll.erase(3);

    // выводим на экран количество удаленных элементов
    cout << "количество удаленных элементов: " << num << endl;

    // выводим на экран все элементы модифицированной коллекции
    copy (coll.cbegin(), coll.cend(),
          ostream_iterator<int>(cout, " "));
    cout << endl;
}
```

Отметим, что контейнеры содержат разные функции-члены `erase()`. Количество удаленных элементов возвращает только вариант, удаляющий элемент по значению, заданному как единственный аргумент (см. раздел 8.7.3). Разумеется, если дубликаты запрещены, то возвращаемое значение может быть равным только 0 или 1. В частности, это относится к контейнерам `set`, `map`, `unordered_set` и `unordered_map`.

Результаты работы программы выглядят следующим образом:

```
1 2 3 4 5 6 7 8 9
количество удаленных элементов: 1
1 2 4 5 6 7 8 9
```

6.7.3. Алгоритмы и функции-члены

Даже если алгоритм формально можно применить, это может оказаться неудачной идеей. Контейнер может иметь функции-члены, обеспечивающие более высокую производительность. Ярким примером является использование функции `remove()` для удаления

элементов из списка. Если применить функцию `remove()` к элементам списка, алгоритм не будет знать, что он работает со списком, и будет работать с ним как с обычным контейнером: переупорядочит элементы, изменив их значения. Например, если алгоритм удалит первый элемент, то все остальные элементы будут присвоены их предшественникам. Это поведение противоречит основному преимуществу списков: возможности вставлять, перемещать и удалять элементы, модифицируя связи, а не значения.

Для того чтобы не снижать производительность, списки имеют специальные функции-члены для всех модифицирующих алгоритмов. Предпочтение всегда следует отдавать функциям-членам. Более того, эти функции-члены действительно удаляют удаленные элементы, как показано в следующем примере:

```
// stl/remove4.cpp

#include <list>
#include <algorithm>
using namespace std;

int main()
{
    list<int> coll;

    // вставляем элементы от 6 до 1 и от 1 до 6
    for (int i=1; i<=6; ++i) {
        coll.push_front(i);
        coll.push_back(i);
    }

    // удаляем все элементы со значением 3 (низкая производительность)
    coll.erase (remove(coll.begin(), coll.end(),
                       3),
               coll.end());

    // удаляем все элементы со значением 4 (высокая производительность)
    coll.remove (4);
}
```

Если целью является высокое быстроедействие программы, то всегда следует отдавать предпочтение функциям-членам, а не алгоритмам. Проблема лишь в том, чтобы знать о существовании функции-члена, обеспечивающей намного более высокую производительность для конкретного контейнера. Если применить алгоритм `remove()` к списку, вы не получите никакого предупреждения или сообщения об ошибке. Однако, если в этой ситуации выбрать функцию-член, то при изменении типа контейнера придется изменять весь код. В главе 11 указано, существует ли функция-член, обеспечивающая более высокую производительность, чем алгоритм.

6.8. Функции в качестве аргументов алгоритма

Для повышения гибкости и мощи программы некоторым алгоритмам можно передавать вспомогательные функции, определенные пользователем. Эти функции вызываются в алгоритмах.

6.8.1. Использование функций в качестве аргументов алгоритмов

Простейший пример — алгоритм `for_each()`, применяющий пользовательскую функцию к каждому элементу в заданном диапазоне. Рассмотрим следующий пример:

```
// stl/foreach1.cpp

#include <vector>
#include <algorithm>
#include <iostream>
using namespace std;

// функция, выводящая на экран передаваемый аргумент
void print (int elem)
{
    cout << elem << ' ';
}

int main()
{
    vector<int> coll;

    // вставляем элементы от 1 до 9
    for (int i=1; i<=9; ++i) {
        coll.push_back(i);
    }

    // выводим на экран все элементы
    for_each (coll.cbegin(), coll.cend(), // диапазон
              print);                     // операция
    cout << endl;
}
```

Алгоритм `for_each()` применяет передаваемую функцию `print()` к каждому элементу в диапазоне `[coll.cbegin(), coll.cend())`. Таким образом, получаем следующий результат работы программы:

```
1 2 3 4 5 6 7 8 9
```

Алгоритмы используют вспомогательные функции в разных вариантах: иногда обязательно, иногда обязательно. В частности, вспомогательные функции можно использовать для того, чтобы задать критерий поиска или сортировки или для определения операции при передаче элементов из одной коллекции в другую.

Рассмотрим еще один пример:

```
// stl/transform1.cpp

#include <set>
#include <vector>
#include <algorithm>
#include <iterator>
```

```

#include <iostream>
#include "print.hpp"

int square (int value)
{
    return value*value;
}

int main()
{
    std::set<int> coll1;
    std::vector<int> coll2;

    // вставляем элементы от 1 до 9 в коллекцию coll1
    for (int i=1; i<=9; ++i) {
        coll1.insert(i);
    }
    PRINT_ELEMENTS(coll1, "инициализированные: ");

    // переносим каждый элемент из коллекции coll1 в коллекцию coll2
    // - возводим в квадрат каждое передаваемое значение
    std::transform (coll1.cbegin(), coll1.cend(), // источник
                    std::back_inserter(coll2),    // назначение
                    square); // операция
    PRINT_ELEMENTS(coll2, "в квадрате: ");
}

```

В данном примере для возведения в квадрат каждого элемента коллекции `coll1`, передаваемого в коллекцию `coll2`, используется функция `square()` (рис. 6.11). Программа выводит на экран следующие строки:

```

инициализированные: 1 2 3 4 5 6 7 8 9
в квадрате:         1 4 9 16 25 36 49 64 81

```

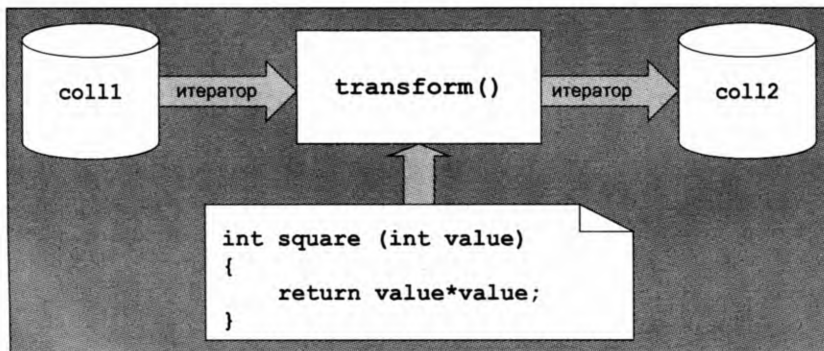


Рис. 6.11. Как работает алгоритм `transform()`

6.8.2. Предикаты

Предикат — это особая разновидность вспомогательной функции. Предикаты возвращают булево значение и часто используются для задания критерия сортировки или поиска. В зависимости от поставленной цели предикаты бывают унарными и бинарными.

Не каждая унарная или бинарная функция, возвращающая булево значение, является корректным предикатом. Кроме того, библиотека STL требует, чтобы предикаты не имели состояния, т.е. они всегда должны возвращать один и тот же результат для одного и того же значения. Это исключает применение функций, модифицирующих свое внутреннее состояние при вызове (см. раздел 10.1.4).

Унарные предикаты

Унарные предикаты проверяют конкретное свойство единственного аргумента. Типичным примером является функция, используемая как критерий поиска для обнаружения первого простого числа.

```
// stl/primel.cpp

#include <list>
#include <algorithm>
#include <iostream>
#include <cstdlib> // для abs()
using namespace std;

// предикат, возвращающий результат проверки, является ли целое число простым
bool isPrime (int number)
{
    // игнорируем отрицательные числа
    number = abs(number);

    // 0 и 1 — не простые числа
    if (number == 0 || number == 1) {
        return false;
    }

    // находим делитель, деление на который происходит без остатка
    int divisor;
    for (divisor = number/2; number%divisor != 0; --divisor) {
        ;
    }

    // если не обнаружены делители больше единицы, значит, число простое
    return divisor == 1;
}

int main()
{
    list<int> coll;

    // вставляем элементы от 24 до 30
    for (int i=24; i<=30; ++i) {
```

```

        coll.push_back(i);
    }

    // ищем простое число
    auto pos = find_if (coll.cbegin(), coll.cend(), // диапазон
                       isPrime);                 // предикат
    if (pos != coll.end()) {
        // найден
        cout << *pos << " – простое число" << endl;
    }
    else {
        // не найден
        cout << "простое число не найдено" << endl;
    }
}

```

В этом примере для поиска первого элемента в заданном диапазоне, для которого передаваемый предикат возвращает true, используется алгоритм `find_if()`. Здесь предикатом является функция `isPrime()`, проверяющая, является ли число простым. Используя этот предикат, алгоритм возвращает первое простое число в заданном диапазоне. Если в диапазоне не найден ни один диапазон, соответствующий предикату, алгоритм возвращает конец диапазона (второй аргумент). Это условие проверяется после выполнения вызова. Коллекция в данном примере содержит простое число в диапазоне от 24 до 30, поэтому результат работы программы выглядит следующим образом:

29 – простое число

Бинарные предикаты

Бинарные предикаты обычно сравнивают конкретное свойство двух аргументов. Например, для сортировки элементов в соответствии с заданным пользователем критерием можно написать простую предикатную функцию. Это может оказаться необходимым в ситуациях, в которых операция `<` неприемлема или когда нужен другой критерий.

В следующем примере сортируются элементы дека по фамилии и имени человека:

```

// stl/sort1.cpp

#include <algorithm>
#include <deque>
#include <string>
#include <iostream>
using namespace std;

class Person {
public:
    string firstname() const;
    string lastname() const;
    ...
};

// бинарный предикат:
// - возвращает результат проверки, предшествует ли один человек другому

```

```
bool personSortCriterion (const Person& p1, const Person& p2)
{
    // человек предшествует другому человеку,
    // - если его фамилия предшествует фамилии другого человека
    // - если фамилии совпадают, а имя первого человека
    // предшествует имени второго
    return p1.lastname() < p2.lastname() ||
        (p1.lastname() == p2.lastname() &&
         p1.firstname() < p2.firstname());
}

int main()
{
    deque<Person> coll;
    ...
    sort(coll.begin(), coll.end(), // диапазон
         personSortCriterion);    // критерий сортировки
    ...
}
```

Отметим, что критерий сортировки можно реализовать как функциональный объект. Преимуществом этого вида реализации является то, что критерием является тип, который можно использовать, например, для объявления множеств, использующих этот критерий для сортировки элементов. Реализация такого критерия сортировки описана в разделе 10.1.1.

6.9. Использование лямбда-выражений

Лямбда-выражения, введенные в стандарте C++11, описывают функциональное поведение в виде выражения или оператора (см. раздел 3.1.10). Благодаря этому можно определять объекты, выражающие функциональное поведение, и передавать их в качестве подставляемых аргументов в алгоритмы для использования в качестве предикатов и для других целей.

Например, рассмотрим такой код:

```
// возводим все элементы в куб
std::transform (coll.begin(), coll.end(), // источник
               coll.begin(),             // назначение
               [](double d) {             // лямбда как функтор
                   return d*d*d;
               });
```

Выражение

```
[](double d) { return d*d*d; }
```

определяет лямбда-выражение, представляющее функциональный объект, возвращающий число типа `double`, возведенное в куб. Как видим, это дает возможность задать функциональное поведение, непосредственно передаваемое алгоритму `transform()` при его вызове.

Преимущества лямбда-функций

Использование лямбда-функций для определения поведения в рамках библиотеки STL устраняет много недостатков прошлых вариантов реализации такой функциональности. Допустим, требуется найти первый элемент в коллекции, имеющий значение больше x и меньше y :

```
// stl/lambda1.cpp

#include <algorithm>
#include <deque>
#include <iostream>
using namespace std;

int main()
{
    deque<int> coll = { 1, 3, 19, 5, 13, 7, 11, 2, 17 };
    int x = 5;
    int y = 12;
    auto pos = find_if (coll.cbegin(), coll.cend(), // диапазон
                        [=](int i) {               // критерий поиска
                            return i > x && i < y;
                        });
    cout << "первый элемент >5 и <12: " << *pos << endl;
}
```

Вызывая алгоритм `find_if()`, мы передаем соответствующий предикат в качестве третьего аргумента.

```
auto pos = find_if (coll.cbegin(), coll.cend(),
                   [=](int i) {
                       return i > x && i < y;
                   });
```

Лямбда-функция — это просто функциональный объект, получающий целое число i и возвращающий результат проверки, больше ли число i числа x и меньше ли оно числа y .

```
[=](int i) {
    return i > x && i < y;
}
```

Указывая `=` в качестве *захвата* внутри конструкции `[=]`, мы передаем в тело лямбда-функции *по значению* символы, которые были корректны в момент объявления лямбда-функции. Таким образом, в лямбда-функции мы можем читать переменные x и y , объявленные в функции `main()`. Если используется конструкция `[&]`, то значения можно передавать по ссылке, так что в теле лямбда-функции эти значения можно модифицировать (детали захвата описаны в разделе 3.1.10).

Сравним теперь этот способ поиска первого элемента, удовлетворяющего условию `>5 и <12`, с другими подходами, использовавшимися в языке C++ до появления лямбда-функций.

- В отличие от циклов, написанных вручную,

```
// первый элемент, удовлетворяющий условию "> x и < y"
vector<int>::iterator pos;
for (pos = coll.begin() ; pos != coll.end(); ++pos) {
    if (*pos > x && *pos < y) {
        break; // the loop
    }
}
```

лямбда-функции позволяют использовать стандартные алгоритмы и избегать неуклюжей конструкции `break`.

- В отличие от предикатов, написанных вручную,

```
bool pred (int i)
{
    return i > x && i < y;
}
...
pos = find_if (coll.begin(), coll.end(), // диапазон
              pred);                  // критерий поиска
```

лямбда-выражения не создают проблем, которые возникают, когда детали поведения определены где-то в другом месте и приходится выяснять, что именно ищет алгоритм `find_if()`, если в программе нет точных комментариев. Кроме того, компиляторы языка C++ лучше оптимизируют лямбда-функции, чем обычные функции.

Еще важнее то, что доступ к переменным `x` и `y` в этом сценарии действительно крайне неудобен. До появления стандарта C++11 обычно в таких ситуациях использовались функциональные объекты (см. раздел 6.10), ярко демонстрирующие неуклюжесть такого подхода:

```
class Pred
{
private:
    int x;
    int y;
public:
    Pred (int xx, int yy) : x(xx), y(yy) {
    }
    bool operator() (int i) const {
        return i > x && i < y;
    }
};
...
pos = find_if (coll.begin(), coll.end(), // диапазон
              Pred(x,y));              // критерий поиска
```

- В отличие от применения связывателей (см. раздел 6.10.3),

```
pos = find_if (coll.begin(), coll.end(), // диапазон
              bind(logical_and<bool>(), // критерий поиска
```

```
bind(greater<int>(), _1, x),
bind(less<int>(), _1, y))));
```

выражения становятся много понятнее.

Итак, лямбда-функции впервые обеспечивают удобный, понятный, быстрый и технологичный подход к использованию алгоритмов STL.

Использование лямбда-функций в качестве критерия сортировки

В качестве другого примера продемонстрируем использование лямбда-выражений для определения критерия сортировки вектора элементов типа `Person` (соответствующая программа, в которой используется функция, определяющая критерий сортировки, приведена в разделе 6.8.2).

```
// stl/sort2.cpp

#include <algorithm>
#include <deque>
#include <string>
#include <iostream>
using namespace std;
class Person {
public:
    string firstname() const;
    string lastname() const;
    ...
};

int main()
{
    deque<Person> coll;
    ...

    // сортируем вектор типа Persons по фамилии (или имени):
    sort(coll.begin(), coll.end(), // диапазон
        [] (const Person& p1, const Person& p2) { // критерий сортировки
            return p1.lastname() < p2.lastname() ||
                (p1.lastname() == p2.lastname() &&
                 p1.firstname() < p2.firstname());
        });
    ...
}
```

Ограничения лямбда-функций

Тем не менее лямбда-функции не всегда являются наилучшим средством. Рассмотрим, например, использование лямбда-функции для определения критерия сортировки для ассоциативных массивов:

```
auto cmp = [] (const Person& p1, const Person& p2) {
    return p1.lastname() < p2.lastname() ||
```

```

        (p1.lastname() == p2.lastname() &&
         p1.firstname() < p2.firstname());
    };
...
std::set<Person, decltype(cmp)> coll(cmp);

```

Поскольку в объявлении объекта класса `set` необходимо определить тип лямбда-функции, приходится использовать операцию `decltype` (см. раздел 3.1.11), возвращающую тип лямбда-объекта, например `cmp`. Отметим, что необходимо также передать лямбда-объект в конструктор `coll`; в противном случае класс `coll` будет вынужден вызвать конструктор по умолчанию для передаваемого критерия сортировки, в то время как лямбда-функции не могут иметь конструкторов по умолчанию и операций присваивания¹¹. Итак, для критерия сортировки класс, определяющий функциональные объекты, может оказаться более интуитивно понятным.

Другая проблема, связанная с лямбда-функциями, заключается в том, что лямбда-функция не может иметь внутреннего состояния, которое могло бы сохраняться при многократных вызовах. Если требуется такое состояние, то необходимо объявить объект или переменную во внешней области видимости и передать ее по ссылке в захват лямбда-функции. По контрасту функциональные объекты позволяют инкапсулировать внутреннее состояние (см. раздел 10.3.2).

Тем не менее лямбда-функции можно использовать для определения хеш-функций и/или критерия эквивалентности для неупорядоченных контейнеров. Пример приведен в разделе 7.9.7.

6.10. Функциональные объекты

Функциональные аргументы алгоритмов не обязаны быть функциями. На примере лямбда-выражений видно, что функциональными аргументами могут быть объекты, ведущие себя как функции. Такие объекты называются *функциональными объектами*, или *функторами*¹². Вместо использования лямбда-функции можно определить функциональный объект как объект класса, содержащего *операцию вызова функции*. Это было возможно еще до появления стандарта C++11.

6.10.1. Определение функциональных объектов

Функциональные объекты — это еще один пример проявления мощи обобщенного программирования и концепции чистой абстракции. Все, что *ведет* себя как функция, является *функцией*. Итак, если определить объект, ведущий себя как функция, то его можно использовать как функцию.

Каково же поведение функции? Функциональное поведение — это нечто, что можно вызывать с помощью пары скобок и передачи аргументов. Например:

¹¹ Благодаря за это замечание Алисдара Мередита (Alisdair Meredith).

¹² В соответствии со стандартом C++11 *функциональным объектом* считается любой объект, который может быть использован в качестве вызова функции. Таким образом, указатели на функции, объекты классов, содержащие операцию `operator()` или преобразование в указатель на функцию, а также лямбда-функции являются функциональными объектами. Однако в книге этот термин применяется только к объектам класса, содержащего оператор `operator()`.

```
function(arg1,arg2); // вызов функции
```

Если требуется, чтобы объекты вели себя подобным образом, необходимо сделать возможным вызов этих объектов с помощью пары скобок и передачи аргументов. Да, это возможно (в языке C++ возможно почти все). Для этого достаточно объявить операцию `()` с соответствующими типами параметров.

```
class X {
public:
    // определение операции "вызов функции":
    return-value operator() (arguments) const;
    ...
};
```

Теперь объекты этого класса можно использовать как вызов функции

```
X fo;
...
fo(arg1,arg2); // вызов operator() из объекта-функции fo
```

Этот вызов эквивалентен конструкции.

```
fo.operator() (arg1,arg2); // вызов operator() из функционального объекта fo
```

Рассмотрим законченный пример. Это вариант функционального объекта из предыдущего примера (см. раздел 6.8.1), который делал то же самое с помощью обычной функции:

```
// stl/foreach2.cpp

#include <vector>
#include <algorithm>
#include <iostream>
using namespace std;

// простой функциональный объект, который выводит на экран
// передаваемые аргументы
class PrintInt {
public:
    void operator() (int elem) const {
        cout << elem << ' ';
    }
};

int main()
{
    vector<int> coll;

    // вставляем элементы от 1 до 9
    for (int i=1; i<=9; ++i) {
        coll.push_back(i);
    }

    // выводим на экран все элементы
    for_each (coll.cbegin(), coll.cend(), // диапазон
```

```

        PrintInt()); // операция
    cout << endl;
}

```

Класс `PrintInt` определяет объекты, к которым можно применять операцию `()` с аргументом типа `int`. Выражение `PrintInt()` в операторе

```
for_each(coll.cbegin(), coll.cend(), PrintInt());
```

создает временный объект этого класса, который передается алгоритму `for_each()` в качестве аргумента.

Алгоритм `for_each()` может быть записан следующим образом:

```

namespace std {
    template <typename Iterator, typename Operation>
    Operation for_each (Iterator act, Iterator end, Operation op)
    {
        while (act != end) { // пока не достигнут конец
            op(*act); // - вызываем op() с текущим элементом
            ++act;    // - перемещаем итератор на следующий элемент
        }
        return op;
    }
}

```

Алгоритм `for_each()` использует временный функциональный объект `op` при вызове `op(*act)` для каждого элемента `act`. Если бы `op` была обычной функцией, то алгоритм `for_each()` просто вызвал бы ее с аргументом `*act`. Если же `op` — функциональный объект, то алгоритм `for_each()` вызывает ее операцию `()` с аргументом `*act`. Таким образом, в этом примере алгоритм `for_each()` выполняет следующий вызов:

```
PrintInt::operator() (*act)
```

Может возникнуть вопрос: зачем все это нужно? Функциональные объекты могут даже показаться странными, уродливыми и бессмысленными. Действительно, они усложняют код. Однако функциональные объекты — это нечто большее, чем просто функции, и благодаря этому они имеют определенные преимущества.

- 1. Функциональный объект — это “функция с состоянием”.** Объекты, ведущие себя как указатели, называются интеллектуальными указателями. Это же относится к объектам, ведущим себя как функции: они могут рассматриваться как “интеллектуальные функции”, потому что обладают дополнительными возможностями помимо операции `()`. Функциональные объекты могут содержать другие функции-члены и атрибуты. Это означает, что функциональные объекты имеют состояние. Фактически одна и та же функциональная возможность, выраженная двумя разными функциональными объектами одного и того же типа, в одно и то же время может иметь разные состояния. Для обычных функций это невозможно. Другим преимуществом функциональных объектов является возможность их инициализации в ходе выполнения программы до вызова.
- 2. Каждый функциональный объект имеет свой тип.** Обычные функции имеют разные типы, только если их сигнатуры отличаются друг от друга. Однако функциональные объекты могут иметь разные типы, даже если их сигнатуры совпадают.

Фактически каждое функциональное поведение, определенное с помощью функционального объекта, имеет свой собственный тип. Это важное улучшение для обобщенного программирования с помощью шаблонов, потому что теперь появляется возможность передавать функциональное поведение в качестве шаблонного параметра. В результате контейнеры разных типов могут использовать один и тот же вид функциональных объектов в качестве критерия сортировки, предотвращая присваивание, объединение и сравнение коллекций, имеющих разные критерии сортировки. Можно даже разрабатывать иерархии функциональных объектов, чтобы, например, создавать конкретные варианты общего критерия.

3. **Функциональные объекты, как правило, работают быстрее, чем обычные функции.** Концепция шаблонов обычно позволяет улучшить оптимизацию, поскольку больше деталей определяется на этапе компиляции. Таким образом, передача функциональных объектов вместо обычных функций часто повышает производительность программы.

В оставшейся части раздела рассматриваются примеры, демонстрирующие преимущество функциональных объектов над обычными функциями. В главе 10, посвященной исключительно функциональным объектам, приведено больше примеров и подробностей. В частности, показано, как извлечь выгоду из возможности передавать функциональное поведение в качестве шаблонного параметра.

Допустим, что мы хотим добавить определенное значение ко всем элементам коллекции. Если это значение известно на этапе компиляции, то можно использовать обычную функцию.

```
void add10 (int& elem)
{
    elem += 10;
}

void f1()
{
    vector<int> coll;
    ...

    for_each (coll.begin(), coll.end(), // диапазон
              add10);                  // операция
}
```

Если необходимо добавлять разные значения, известные на этапе компиляции, то можно использовать шаблон.

```
template <int theValue>
void add (int& elem)
{
    elem += theValue;
}

void f1()
{
    vector<int> coll;
    ...
}
```

```

for_each (coll.begin(), coll.end(), // диапазон
          add<10>);                // операция
}

```

Если значение добавляется на этапе выполнения программы, ситуация усложняется. В этом случае необходимо передавать значение в функцию до того, как она будет вызвана. Как правило, для этого используется глобальная переменная, которая используется как функцией, вызывающей алгоритм, так и функцией, которая вызывается алгоритмом, добавляющим значение. Это плохой стиль программирования.

Если такая функция потребуется дважды, чтобы добавить два разных значения, и оба этих значения должны добавляться на этапе выполнения программы, одной обычной функцией уже не обойтись. Придется либо передать дескриптор, либо написать две разные функции. Вы копировали когда-нибудь функцию, содержащую статическую переменную для хранения ее состояния, просто потому, что вам потребовалась точно такая же функция с другим состоянием в тот же момент времени? Эта проблема относится к той же категории.

Работая с функциональными объектами, можно написать более интеллектуальную функцию, ведущую себя желательным образом. Поскольку она может иметь состояние, объект может быть инициализирован правильным значением. Рассмотрим законченный пример¹³:

```

// stl/add1.cpp

#include <list>
#include <algorithm>
#include <iostream>
#include "print.hpp"
using namespace std;

// функциональный объект, добавляющий значение, полученное при инициализации
class AddValue {
private:
    int theValue; // добавляемое значение
public:
    // конструктор инициализирует добавляемое значение
    AddValue(int v) : theValue(v) {
    }

    // "вызов функции" для элемента, к которому добавляется значение
    void operator() (int& elem) const {
        elem += theValue;
    }
};

int main()
{
    list<int> coll;

    // вставляем элементы от 1 до 9

```

¹³ Вспомогательная функция PRINT_ELEMENTS() была введена в разделе 6.6.

```
for (int i=1; i<=9; ++i) {
    coll.push_back(i);
}

PRINT_ELEMENTS(coll, "инициализирован: ");

// добавляем значение 10 к каждому элементу
for_each (coll.begin(), coll.end(), // диапазон
          AddValue(10));           // операция

PRINT_ELEMENTS(coll, "после добавления 10: ");

// добавляем значение первого элемента к каждому элементу
for_each (coll.begin(), coll.end(), // диапазон
          AddValue(*coll.begin())); // операция

PRINT_ELEMENTS(coll, "после добавления первого элемента: ");
}
```

После инициализации коллекция содержит значения от 1 до 9:

инициализирован: 1 2 3 4 5 6 7 8 9

Первый вызов алгоритма `for_each()` добавляет 10 к каждому значению:

```
for_each (coll.begin(), coll.end(), // диапазон
          AddValue(10));           // операция
```

Здесь выражение `AddValue(10)` создает объект типа `AddValue`, инициализированный значением 10. Конструктор `AddValue` сохраняет это значение как член `theValue`. В алгоритме `for_each()` вызывается операция `()` для каждого элемента коллекции `coll`. Этот вызов операции `()` относится к передаваемому временному функциональному объекту типа `AddValue`. Текущий элемент передается как аргумент.

Функциональный объект добавляет свое значение 10 к каждому элементу. После этого элементы принимают следующие значения:

после добавления 10: 11 12 13 14 15 16 17 18 19

Второй вызов алгоритма `for_each()` использует ту же самую функциональную возможность для добавления значения первого элемента ко всем элементам коллекции. Этот вызов инициализирует временный функциональный объект типа `AddValue` первым элементом коллекции.

```
AddValue(*coll.begin())
```

Результат работы программы принимает следующий вид:

после добавления первого элемента: 22 23 24 25 26 27 28 29 30

Усовершенствованный вариант этого примера, в котором тип функционального объекта `AddValue` является шаблонным типом добавляемого значения, рассматривается в разделе 11.4.

Используя этот прием, можно решить проблему с помощью двух разных функциональных объектов, имеющих разные состояния в один и тот же момент времени. Например, можно просто объявить функциональные объекты и использовать их независимо друг от друга.

```
AddValue addx(x); // функциональный объект, добавляющий значение x
AddValue addy(y); // функциональный объект, добавляющий значение y
for_each (coll.begin(),coll.end(), // добавляем значение x к каждому элементу
          addx);

...
for_each (coll.begin(),coll.end(), // добавляем значение y к каждому элементу
          addy);

...
for_each (coll.begin(),coll.end(), // добавляем значение x к каждому элементу
          addx);
```

Аналогично можно предусмотреть дополнительную функцию-член для запроса или изменения состояния функционального объекта на протяжении срока его существования. Хороший пример приведен в разделе 10.1.3.

Отметим, что для некоторых алгоритмов стандартная библиотека языка не регламентирует, как часто функциональные объекты могут вызываться для каждого элемента, и может так случиться, что элементам будут передаваться разные копии функционального объекта. Если функциональные объекты используются как предикаты, это может вызвать ужасные последствия. Эта тема освещается в разделе 10.1.4.

6.10.2. Стандартные функциональные объекты

Стандартная библиотека языка C++ содержит несколько стандартных функциональных объектов, выполняющих основные операции. Благодаря им в определенных ситуациях можно обойтись без создания собственных функциональных объектов. Типичным примером является функциональный объект, используемый как критерий сортировки. Критерий сортировки с помощью операции < представляет собой стандартный функциональный объект `less<>`. Таким образом, объявление

```
set<int> coll;
```

разворачивается в объявление

```
set<int,less<int>> coll; // сортируем элементы с помощью операции <
```

Теперь легко упорядочить элементы в обратном порядке.

```
set<int,greater<int>> coll; // сортируем элементы с помощью операции >
```

Другим примером применения стандартных функциональных объектов являются алгоритмы. Рассмотрим следующую программу:

```
// stl/fo1.cpp

#include <deque>
#include <algorithm>
#include <functional>
```

```
#include <iostream>
#include "print.hpp"
using namespace std;

int main()
{
    deque<int> coll = { 1, 2, 3, 5, 7, 11, 13, 17, 19 };

    PRINT_ELEMENTS(coll, "инициализация: ");

    // меняем знаки всех значений в коллекции coll на противоположный
    transform (coll.cbegin(), coll.cend(), // источник
               coll.begin(),              // назначение
               negate<int>());             // операция
    PRINT_ELEMENTS(coll, "смена знака: ");

    // возводим все значения в коллекции coll в квадрат
    transform (coll.cbegin(), coll.cend(), // первый источник
               coll.cbegin(),              // второй источник
               coll.begin(),               // назначение
               multiplies<int>());          // операция
    PRINT_ELEMENTS(coll, "в квадрате: ");
}
```

Сначала в программу включается заголовок для стандартных объектов-функций: `<functional>`

```
#include <functional>
```

Затем два стандартных функциональных объекта используются для замены знака и возведения в квадрат элементов коллекции `coll`. Во фрагменте

```
transform (coll.cbegin(), coll.cend(), // источник
           coll.begin(),              // назначение
           negate<int>());             // операция
```

выражение

```
negate<int>()
```

создает объект-функцию стандартного шаблонного класса `negate<>`, который просто возвращает элемент типа `int`, имеющий противоположный знак по отношению к тому элементу, для которого он был вызван. Алгоритм `transform()` использует эту операцию для преобразования всех элементов первой коллекции во вторую. Если диапазоны источника и назначения совпадают, как в данном случае, возвращаемые элементы с противоположным знаком заменяют исходные элементы. Таким образом, данный оператор изменяет на противоположные знаки всех элементов коллекции.

Аналогично функциональный объект `multiplies` используется для возведения в квадрат всех элементов коллекции `coll`.

```
transform (coll.cbegin(), coll.cend(), // первый источник
           coll.cbegin(),              // второй источник
```

```
coll.begin(),           // назначение
multiplies<int>());     // операция
```

Здесь другая форма алгоритма `transform()` объединяет элементы двух коллекций с помощью заданной операции и записывает результат в третью коллекцию. Как и раньше, все коллекции совпадают, поэтому каждый элемент умножается на себя, а результаты заменяют старые значения.

Итак, программа выводит на экран следующие строки:

```
инициализация: 1 2 3 5 7 11 13 17 19
смена знака:   -1 -2 -3 -5 -7 -11 -13 -17 -19
в квадрате:    1 4 9 25 49 121 169 289 361
```

6.10.3. Связыватели

Для объединения функциональных объектов с другими значениями и выполнения других специфических задач используются *функциональные адаптеры*, или *связыватели* (binders). Рассмотрим законченный пример.

```
// stl/bind1.cpp

#include <set>
#include <deque>
#include <algorithm>
#include <iterator>
#include <functional>
#include <iostream>
#include "print.hpp"
using namespace std;
using namespace std::placeholders;

int main()
{
    set<int,greater<int>> > coll1 = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
    deque<int> coll2;

    // Примечание: благодаря критерию сортировки greater<>()
    // элементы следуют в обратном порядке:
    PRINT_ELEMENTS(coll1,"инициализация: ");

    // преобразовываем все элементы в коллекцию coll2,
    // умножая их на 10
    transform (coll1.cbegin(),coll1.cend(),           // источник
               back_inserter(coll2),                 // назначение
               bind(multiplies<int>(),_1,10));        // операция
    PRINT_ELEMENTS(coll2,"преобразование: ");

    // заменяем значение, равное 70, значением, равным 42
    replace_if (coll2.begin(),coll2.end(),           // диапазон
               bind(equal_to<int>(),_1,70),         // диапазон замены
               42);                                   // новое значение
    PRINT_ELEMENTS(coll2,"замена: ");
```

```
// удаляем все элементы со значениями от 50 до 80
coll2.erase(remove_if(coll2.begin(), coll2.end(),
    bind(logical_and<bool>(),
    bind(greater_equal<int>(), _1, 50),
    bind(less_equal<int>(), _1, 80))),
    coll2.end());
PRINT_ELEMENTS(coll2, "удаление: ");
}
```

Здесь оператор

```
transform (coll1.cbegin(), coll1.cend(),    // источник
    back_inserter(coll2),                // назначение
    bind(multiplies<int>(), _1, 10)); // операция
```

преобразовывает все элементы коллекции `coll1` в коллекцию `coll2` (вставляя их), умножая каждый элемент на 10. Для определения соответствующей операции используется функциональный адаптер `bind()`, позволяющий создавать высокоуровневые функциональные объекты из низкоуровневых функциональных объектов и шаблонных заготовок, представляющих собой числовые идентификаторы, начинающиеся с символа подчеркивания. Оператор

```
bind(multiplies<int>(), _1, 10)
```

определяет функциональный объект, умножающий первый переданный аргумент на 10.

Аналогичный функциональный объект можно было бы использовать для умножения на 10 всех элементов. Например, следующие инструкции записывают в стандартный поток вывода число 990:

```
auto f = bind(multiplies<int>(), _1, 10);
cout << f(99) << endl;
```

Этот функциональный объект передается алгоритму `transform()`, который ожидает в качестве четвертого аргумента операцию, получающую один аргумент, а именно текущий элемент. Впоследствии алгоритм `transform()` выполняет операцию “умножить на 10” для каждого элемента и вставляет результат в коллекцию `coll2`, т.е. после всех элементов коллекции `coll2` следуют все значения коллекции `coll1`, умноженные на 10.

Аналогично в вызове

```
replace_if (coll2.begin(), coll2.end(),    // диапазон
    bind(equal_to<int>(), _1, 70),        // критерий замены
    42);                                  // новое значение
```

в качестве критерия для выбора элемента, который должен быть заменен числом 42, используется следующий функциональный адаптер.

```
bind(equal_to<int>(), _1, 70)
```

Здесь функциональный адаптер `bind()` вызывает бинарный предикат `equal_to`, которому в качестве первого параметра передается первый аргумент, а в качестве второго параметра — число 70. Таким образом, функциональный объект, определенный с помощью связывателя `bind()`, возвращает `true`, если переданный аргумент (элемент коллекции `coll2`) равен 70. В результате оператор заменяет все значения, равные 70, значением 42.

Последний пример использует комбинацию связывателей, где оператор

```
bind(logical_and<bool>(),
    bind(greater_equal<int>(), _1, 50),
    bind(less_equal<int>(), _1, 80))
```

задает для параметра *x* унарный предикат “*x* ≥ 50 & *x* ≤ 80.” Этот пример демонстрирует возможность использования вложенных связывателей `bind()` для описания более сложных предикатов и функциональных объектов. В данном случае алгоритм `remove_if()` использует функциональный объект для удаления из коллекции всех значений, лежащих в диапазоне от 50 до 80. Фактически алгоритм `remove_if()` лишь изменяет порядок и возвращает новый конец диапазона, а функция-член `coll2.erase()` удаляет “удаленные” элементы из коллекции `coll2` (см. детали в разделе 6.7.1).

Результат работы программы выглядит следующим образом:

```
инициализация:  9 8 7 6 5 4 3 2 1
преобразование: 90 80 70 60 50 40 30 20 10
замена:         90 80 42 60 50 40 30 20 10
удаление:       90 42 40 30 20 10
```

Отметим, что заполнители имеют свое пространство имен: `std::placeholders`. По этой причине в начало программы вставлена соответствующая директива, позволяющая задавать в качестве первого или второго параметра связывателя заполнители `_1` или `_2`. Без использования этой директивы связыватели пришлось бы определять следующим образом:

```
std::bind(std::logical_and<bool>(),
    std::bind(std::greater_equal<int>(), std::placeholders::_1, 50),
    std::bind(std::less_equal<int>(), std::placeholders::_1, 80))
```

Этот вид программирования называется *функциональной композицией* (functional composition). Интересно, что все эти функциональные объекты обычно объявляются встраиваемыми. Таким образом, мы используем функциональную абстракцию и при этом получаем высокую производительность.

Существуют и другие способы определения функциональных объектов. Например, для вызова функции-члена для каждого элемента коллекции можно использовать вызов

```
for_each(coll.cbegin(), coll.cend(), // диапазон
    bind(&Person::save, _1)); // операция: Person::save(elem)
```

Функциональный объект `bind` связывает указанную функцию-член, чтобы она вызывалась для каждого элемента, который передается с помощью заполнителя `_1`. Таким образом, для каждого элемента коллекции `coll` вызывается функция-член `save()` из класса `Person`. Разумеется, все это работает только при условии, что элементы имеют тип `Person` или тип, производный от типа `Person`.

Более подробно все стандартные функциональные объекты, функциональные адаптеры и аспекты функциональной композиции рассматриваются в разделе 10.2. Кроме того, там же объясняется, как написать свои собственные функциональные объекты.

До появления документа TR1 для функциональной композиции использовались другие связыватели и адаптеры: `bind1st()`, `bind2nd()`, `ptr_fun()`, `mem_fun()` и `mem_fun_ref()`, которые в стандарте C++11 объявлены устаревшими. Детали описаны в разделе 10.2.4.

6.10.4. Функциональные объекты и связыватели против лямбда-функции

Лямбда-функции — это разновидность неявно определенного функционального объекта. Как указано в разделе 6.9, лямбда-функции обычно обеспечивают более интуитивный подход к определению функционального поведения алгоритмов из библиотеки STL. Кроме того, лямбда-функции работают быстрее функциональных объектов.

Тем не менее у лямбда-функций есть несколько недостатков.

- У такого функционального объекта нет скрытого внутреннего состояния. Вместо этого все данные, определяющие его состояние, определяются вызывающей функцией и передаются в виде захвата.
- Исчезает преимущество частичного описания функционального поведения, которое требуется в разных местах. Вы можете определить лямбда-функцию, а затем присвоить ее объекту `auto` (см. раздел 6.9), но читабельность такого определения довольно сомнительна.

6.11. Элементы контейнеров

Элементы контейнеров должны удовлетворять определенным требованиям, поскольку контейнеры обрабатывают их определенным образом. В этом разделе мы опишем эти требования и обсудим последствия того факта, что контейнеры неявно создают копии своих элементов.

6.11.1. Требования к элементам контейнеров

Контейнеры, итераторы и алгоритмы библиотеки STL являются шаблонами. Вследствие этого они могут работать как со стандартными, так и с пользовательскими типами. Однако операции, которые они вызывают, накладывают на них определенные ограничения. Элементы контейнеров STL должны удовлетворять трем основным требованиям.

1. Элемент должен допускать *копирование* или *перемещение*. Таким образом, тип элемента явно или неявно должен иметь копирующий или перемещающий конструктор.
2. Сгенерированная копия должна быть эквивалентной оригиналу. Это значит, что любая проверка равенства должна возвращать одинаковые результаты как для оригинала, так и для копии.
3. Элемент должен допускать (*перемещающее*) *присваивание*. Контейнеры и алгоритмы используют операции присваивания для перезаписи старых элементов новыми.
4. Элемент должен допускать *удаление* с помощью деструктора. Контейнеры должны удалять внутренние копии элементов, когда эти элементы удаляются из контейнера. Таким образом, деструктор не должен быть закрытым. Кроме того, как обычно в языке C++, деструктор не должен генерировать исключение; в противном случае возникает неопределенное поведение.

Эти три операции неявно генерируются для любого класса. Таким образом, класс автоматически удовлетворяет перечисленным требованиям, если в нем не определены

специальные версии этих операций и нет специальных функций-членов, нарушающих правильную работу этих операций.

Элементы контейнеров должны также удовлетворять следующим требованиям.

- Для некоторых функций-членов последовательных контейнеров должен быть доступным *конструктор, заданный по умолчанию*. Например, можно создать непустой контейнер или увеличить количество элементов в контейнере, не имея представления о том, какие значения должны принимать новые значения. Эти элементы создаются без аргументов с помощью вызова конструктора, заданного по умолчанию для данного типа.
- Для некоторых операций необходимо определять *проверку равенства* с помощью операции `==`. Это особенно необходимо при поиске элементов. Однако для неупорядоченных контейнеров можно предусмотреть собственное определение эквивалентности, если их элементы не поддерживают операцию `==` (см. раздел 7.9.7).
- Элементы ассоциативных контейнеров должны поддерживать операции *критерии сортировки*. По умолчанию в качестве таковой используется операция `<`, которая вызывается функциональным объектом `less<>`.
- Элементы неупорядоченных контейнеров должны предусматривать *хеш-функцию и критерий эквивалентности*. Детали описаны в разделе 7.9.2.

6.11.2. Семантика значений и семантика ссылок

Как правило, все контейнеры создают внутренние копии своих элементов и возвращают их, а не оригиналы. Это означает, что элементы контейнера равны, но не идентичны объектам, помещенным в контейнер. Если модифицировать объект как элемент контейнера, то изменения коснутся лишь копии, а не оригинала.

Копирование объектов означает, что контейнеры STL реализуют *семантику значений*. Контейнеры содержат значения вставленных объектов, а не сами объекты. Однако на практике нам часто нужна *семантика ссылок*. Это значит, что контейнеры содержат ссылки на объекты и именно ссылки являются их элементами.

Ориентация библиотеки STL исключительно на семантику значений имеет как преимущества, так и недостатки. Перечислим преимущества.

- Копирование элементов выполняется просто.
- Ссылки провоцируют ошибки. Постоянно необходимо следить за тем, чтобы ссылки, ссылающиеся на несуществующий объект, уничтожались. Требуется также уметь справляться с циклическими ссылками, которые могут возникнуть.

Недостатки описаны ниже.

- Копирование элементов может снизить производительность работы программы или вообще быть невозможным.
- Управление одним и тем же объектом в разных контейнерах в одно и то же время невозможно.

На практике необходимы оба подхода; нужны и копии, не зависящие от оригиналов (семантика значений), и копии, ссылающиеся на оригинал и соответственно изменяющиеся (семантика ссылок). К сожалению, стандартная библиотека C++ не поддерживает

семантику ссылок. Однако семантику ссылок можно реализовать самостоятельно в терминах семантики значений.

Очевидный подход к реализации семантики ссылок — использование в качестве элементов контейнеров указателей¹⁴. Однако обычные указатели порождают обычные проблемы. Например, объект, на который он указывает, может больше не существовать, а сравнения могут оказаться некорректными из-за того, что сравниваются указатели, а не объекты. Таким образом, при использовании указателей в качестве элементов контейнера следует быть очень осторожным. Лучше использовать *интеллектуальные указатели*: объекты, имеющие интерфейс указателей, и дополнительные внутренние операции проверки или обработки данных. С появлением документа TR1 стандартная библиотека C++ содержит класс `shared_ptr` для интеллектуальных указателей, которые могут ссылаться на один и тот же объект (см. раздел 5.2.1). Кроме того, можно использовать класс `std::reference_wrapper<>` (см. раздел 5.4.3), позволяющий контейнерам STL содержать ссылки. Примеры обоих подходов описаны в разделе 7.11.

6.12. Ошибки и исключения в библиотеке STL

Ошибки случаются. Они могут быть логическими ошибками, совершенными программистом, или ошибками времени выполнения, вызванными контекстом или средой (например, из-за нехватки памяти). Оба вида этих ошибок можно обработать с помощью исключений. В этом разделе обсуждаются ошибки и исключения, обрабатываемые в библиотеке STL.

6.12.1. Обработка ошибок

Цель разработки библиотеки STL — обеспечить высокую производительность, а не наивысшую безопасность. Обработка ошибок требует времени, поэтому в библиотеке STL она почти не предусмотрена. Это прекрасно, если в программе нет никаких ошибок, и чревато катастрофой, если это не так. Еще когда библиотека STL не была включена в состав стандартной библиотеки C++, велись дискуссии о том, следует ли включать в нее обработку ошибок. Большинство программистов решили, что обработку ошибок включать в библиотеку не следует, ссылаясь на две причины.

1. Проверка ошибок уменьшает производительность, а скорость — главная цель программ. Как уже указывалось, высокая производительность — одна из основных целей разработки библиотеки STL.
2. Если безопасность важнее, чем быстродействие, ее можно обеспечить, либо добавив оболочки, либо используя специальные версии библиотеки STL. Однако, встраивая проверку ошибок во все основные операции, вы снижаете быстродействие программ. Например, если каждая операция по вычислению индекса будет проверять, не выходит ли он за пределы допустимого диапазона, то не сможете написать обращение к элементам массива без проверки этой ошибки. Однако возможно наличие обходного пути.

¹⁴ Программистам на языке C знаком способ реализации семантики ссылок с помощью указателей. В языке C аргументы функции можно передавать только по значению, поэтому для вызова по ссылке используются указатели.

В итоге проверку ошибок в библиотеке STL признали возможной, но не обязательной.

В основу стандартной библиотеки C++ положен следующий принцип: любое нарушение библиотекой STL заданных предусловий приводит к неопределенному поведению. Таким образом, если индексы, итераторы или диапазоны оказываются некорректными, то результат считается неопределенным. Если вы не используете безопасную версию библиотеки STL, то в результате нередко возникает попытка доступа к неопределенной памяти, порождающая ужасные последствия и даже крах программы. В этом смысле библиотека уязвима для ошибок, как указатели в языке C. Поиск таких ошибок может оказаться очень сложным, особенно если не используется безопасная версия библиотеки STL.

В частности, использование библиотеки STL требует выполнения следующих условий.

- Итераторы должны быть корректными. Например, перед использованием они должны быть инициализированы. Отметим, что итераторы могут стать некорректными в результате побочного эффекта от других операций. В частности, итераторы становятся некорректными
 - при вставке, удалении или перемещении элементов векторов и деков;
 - при повторном хешировании неупорядоченных контейнеров (которое также может быть результатом вставки).
- Итераторы, ссылающиеся на запредельную позицию, не ссылаются ни на один конкретный элемент. Таким образом, выполнение операций `*` или `->` для них запрещено. Особенно это относится к значениям, возвращаемым функциями-членами контейнеров `end()`, `cend()` и `rend()`.
- Диапазоны должны быть корректными.
 - Оба итератора, определяющие диапазон, должны ссылаться на один и тот же контейнер.
 - Второй итератор должен быть достижимым из первого.
- Если используется несколько источников, то второй и последующие должны иметь не меньше элементов, чем первый.
- Диапазоны назначения должны иметь достаточное количество элементов, которые можно заменить; в противном случае необходимо использовать итераторы вставки.

Следующий пример демонстрирует несколько возможных ошибок:

```
// stl/iterbug.cpp

#include <vector>
#include <algorithm>
using namespace std;

int main()
{
    vector<int> coll1; // пустая коллекция
    vector<int> coll2; // пустая коллекция

    // ОШИБКА ВО ВРЕМЯ ВЫПОЛНЕНИЯ ПРОГРАММЫ:
    // - начало расположено за концом диапазона
    vector<int>::iterator pos = coll1.begin();
    reverse (++pos, coll1.end());
```

```

// вставляем элементы от 1 до 9 в коллекцию coll1
for (int i=1; i<=9; ++i) {
    coll1.push_back (i);
}

// ОШИБКА ВО ВРЕМЯ ВЫПОЛНЕНИЯ ПРОГРАММЫ:
// - замена несуществующих элементов
copy (coll1.cbegin(), coll1.cend(), // источник
      coll2.begin());              // назначение

// ОШИБКА ВО ВРЕМЯ ВЫПОЛНЕНИЯ ПРОГРАММЫ:
// - ошибочные коллекции
// - cbegin() и cend() ссылаются на разные коллекции
copy (coll1.cbegin(), coll2.cend(), // источник
      coll1.end());                 // назначение
}

```

Отметим, что, поскольку эти ошибки возникают во время выполнения программы, а не на этапе компиляции, последствия остаются непредсказуемыми.

Существует много способов сделать ошибки при работе с библиотекой STL, причем она никак не пытается предотвратить эти ошибки. Таким образом, целесообразно использовать безопасную библиотеку STL, по крайней мере на этапе разработки программного обеспечения. Первая версия безопасной библиотеки STL была разработана Кеєм Хорстманом (Caу Horstmann) [SafeSTL]. Другим примером является библиотека “STLport”, доступная без ограничений практически на любой платформе [STLport]. Кроме того, поставщики библиотек стали предусматривать флаги для включения безопасного режима, особенно полезного на этапе разработки программ¹⁵.

6.12.2. Обработка исключений

Библиотека почти никогда не проверяет логические ошибки. Вследствие этого логические проблемы почти никогда не заставляют библиотеку STL генерировать исключения. Фактически только две функции в соответствии со стандартом могут генерировать исключение непосредственно: функция-член `at()`, реализующая версию операции индексирования с проверкой, и функция-член `reserve()`, генерирующая исключение, если переданное количество элементов превышает значение, возвращаемое функцией-членом `max_size()`. В остальных ситуациях стандарт требует лишь, чтобы генерировались обычные стандартные исключения, такие как `bad_alloc`, из-за нехватки памяти или исключения, предусмотренные для пользовательских операций.

Когда генерируются исключения и что при этом происходит с компонентами библиотеки STL? В течение продолжительного процесса выработки стандарта C++98 эти вопросы оставались без ответа. Фактически каждое исключение приводило к неопределенному поведению. Даже уничтожение контейнера STL приводило к таковому, если исключение генерировалось во время выполнения одной из операций. Таким образом, библиотека STL была бесполезной, если программисту требовались гарантии и определенное поведение, поскольку не было возможным даже раскручивание стека.

¹⁵ Например, компилятор `gcc` для этой цели предусматривает ключ `-D_GLIBCXX_DEBUG`.

Способ обработки исключений был одним из последних тем, рассмотренных в процессе выработки стандарта C++98. Найти хорошее решение было непросто, и его поиск занял много времени по следующим причинам.

1. Было очень трудно определить степень безопасности, которую должна обеспечить стандартная библиотека C++. Может показаться, что в любом случае следует стремиться к наивысшей степени безопасности. Например, можно предположить, что вставка нового элемента в любую позицию вектора должна быть либо успешной, либо нет. Обычно исключение может возникнуть при копировании элементов вправо, чтобы создать место для нового элемента, и при этом полное восстановление невозможно. Для того чтобы достичь поставленной цели, операция вставки должна выполнить копирование *каждого* элемента вектора в новое хранилище, что значительно снизит производительность программы. Если целью проекта является производительность, как в случае библиотеки STL, идеальную обработку ошибок реализовать не удастся ни при каких условиях. Необходимо искать компромисс между безопасностью и быстродействием.
2. Нет никаких сомнений, что лучше иметь гарантированное, точно определенное поведение исключений без снижения быстродействия, чем рисковать крахом системы. Однако обработка исключений снижает производительность программы. Это может противоречить основной цели проекта — обеспечить максимальное быстродействие. В процессе выработки стандарта C++98 разработчики компилятора утверждали, что в принципе обработку исключений можно реализовать без значительного снижения производительности. Однако оказалось, что спецификации исключений все же снижают производительность, поэтому в стандарте C++11 они были заменены ключевым словом `noexcept` (см. раздел 3.1.7).

В результате всех этих обсуждений стандартная библиотека C++, соответствующая стандарту, дает следующую основную гарантию безопасности исключений: при генерировании исключений стандартная библиотека C++ не порождает утечек ресурсов и не нарушает инварианты контейнеров¹⁶.

К сожалению, во многих ситуациях этого недостаточно. Часто необходимо иметь более сильные гарантии, чем то, что при генерировании исключения операция не будет иметь последствий. С точки зрения исключений такие операции можно рассматривать как *атомарные*. Разумеется, применяя терминологию баз данных, мы могли бы сказать, что эти операции поддерживают принцип *фиксации или отката* (`commit-or-rollback`), или *безопасность транзакций* (`transaction safe`).

С учетом этой более сильной гарантии стандартная библиотека C++ обеспечивает следующее.

- Функции `erase()`, `clear()`, `pop_back()`, `pop_front()` или `swap()` в общем случае не генерируют исключений. Кроме того, ни копирующие конструкторы, ни операции присваивания возвращаемого итератора не генерируют исключений.
- Для всех *контейнеров на основе узлов* (списков, односвязных списков, множеств, мультимножеств, отображений и мультиотображений), включая *неупорядоченные контейнеры*, при любом сбое во время создания узла контейнер остается в текущем

¹⁶ Я выражаю горячую благодарность Дэйву Абрамсу (Dave Abrahams) и Грэгу Колвину (Greg Colvin) за их работу по обеспечению безопасности исключений в стандартной библиотеке C++ и сделанные ими ценные замечания.

состоянии. Более того, удаление узла не может приводить к сбою, при условии, что деструкторы не генерируют исключений. Однако при выполнении операции вставки нескольких элементов в ассоциативные контейнеры необходимость сохранения порядка следования элементов делает полное восстановление после генерации исключения непрактичным. Таким образом, все операции вставки единственного элемента в ассоциативные или неупорядоченные контейнеры подчиняются принципу фиксации и отката, при условии, что хеш-функции неупорядоченных контейнеров не генерируют исключений. Иначе говоря, вставка единственного элемента либо оказывается успешной, либо не имеет последствий. Кроме того, гарантируется, что все операции удаления как единственного элемента, так и нескольких элементов одной операцией всегда являются успешными, при условии, что функции сравнения и хеш-функция контейнера не генерируют исключений.

- Для списков даже операции вставки нескольких элементов обеспечивают безопасность транзакций. Фактически все операции над списком, за исключением `remove()`, `remove_if()`, `merge()`, `sort()` и `unique()`, либо оказываются успешными, либо не имеют последствий. Для некоторых исключительных ситуаций стандартная библиотека C++ предусматривает условные гарантии. Таким образом, если вам нужен контейнер, обеспечивающий безопасность транзакций, следует выбирать список.
- Для односвязных списков функции-члены `insert_after()`, `emplace_after()` и `push_front()` обеспечивают безопасность транзакций¹⁷.
- Все *контейнеры на основе массивов* (объекты класса `array`, векторы и деки) не могут полностью восстановить свое состояние после неудачной операции вставки. Для этого пришлось бы перед выполнением любой операции вставки копировать все последующие элементы, что заняло бы много времени. В то же время операции `push` и `pop`, работающие с последним элементом, не требуют копирования существующих элементов. Если они генерируют исключение, то гарантируется, что последствий не будет. Однако, если перемещающий конструктор не гарантирует отсутствие исключений, то для обеспечения такого поведения следует использовать копирующий, а не перемещающий конструктор. Таким образом, спецификации `nothrow` и `noexcept` для операций, перемещающих элементы, повышают быстродействие программы. Более того, если элементы имеют тип, содержащий операции копирования (копирующий конструктор и операцию присваивания), не генерирующие исключений, то любая операция контейнера для этих элементов либо будет успешной, либо не будет иметь последствий.

Отметим, что все эти гарантии основаны на требовании, чтобы деструкторы никогда не генерировали исключения. Стандартная библиотека C++ строго выполняет это требование, и точно так же должны поступать программисты.

Если необходим контейнер, полностью реализующий принцип фиксации и отката, следует выбирать либо список (не вызывая функции `remove()`, `remove_if()`, `merge()`, `sort()` и `unique()`), либо ассоциативный или неупорядоченный контейнер (не вызывая операции одновременной вставки нескольких элементов). Это позволяет избежать создания копий перед выполнением модифицирующей операции, чтобы предотвратить потерю данных. Отметим, что создание копий контейнера может быть очень затратным.

¹⁷ Стандарт C++11 не утверждает этого относительно функций-членов `emplace_after()` и `push_front()`, но это, скорее всего, просто недоразумение.

Если использовать контейнер на основе узлов невозможно и в то же время необходимо реализовать принцип фиксации и отката, следует предусмотреть оболочки для каждой критичной операции. Например, следующая функция почти безопасно вставляет значение в указанную позицию любого контейнера:

```
template <typename T, typename Cont, typename Iter>
void insert (Cont& coll, const Iter& pos, const T& value)
{
    Cont tmp(coll);                // копируем контейнер и все элементы
    try {
        coll.insert(pos,value);    // пытаемся модифицировать копию сору
    }
    catch (...) {                  // в случае исключения
        coll.swap(tmp);            // - восстанавливаем исходный контейнер
        throw;                     // - и повторно генерируем исключение
    }
}
```

Следует обратить внимание на слово “почти”, потому что эта функция не идеальна: операция `swap()` генерирует исключение, если операция копирования критерия сравнения для ассоциативного контейнера генерирует исключение. Как видим, обработка исключений — действительно нелегкое дело.

6.13. Расширение библиотеки STL

Библиотека STL представляет собой каркас, который можно расширять в любом направлении.

6.13.1. Интеграция дополнительных типов

Программист может создавать свои собственные контейнеры, итераторы, алгоритмы или функциональные объекты, при условии, что они удовлетворяют определенным требованиям. Собственно, в стандартной библиотеке C++ нет некоторых полезных расширений. Это сделано потому, что комитет остановил включение новых средств и сосредоточился на усовершенствовании существующих компонентов, иначе работа никогда не была бы закончена. По этой причине, например, хеш-таблицы не стали частью стандарта C++98.

В качестве полезных расширений можно назвать итераторы (разделы 9.6 и 14.3), контейнеры (раздел 7.10) и алгоритмы (разделы 7.6.2 и 9.5.1). Отметим, что все эти расширения следуют принципам обобщенного программирования.

- Все, что *ведет* себя как контейнер, *является* контейнером.
- Все, что *ведет* себя как итератор, *является* итератором.

Таким образом, работая с контейнерным классом, можно интегрировать его в библиотеку STL, снабдив соответствующим интерфейсом (`begin()`, `end()` и некоторыми определениями типов и т.д.). Если невозможно добавить функции-члены в такой класс, то можно создать класс-оболочку, обеспечивающий соответствующие итераторы.

Отметим, тем не менее, что некоторые контейнерные объекты не соответствуют концепции библиотеки STL. Например, тот факт, что контейнеры STL имеют начало и конец,

затрудняет создание кольцевых контейнеров, таких как кольцевой буфер, в рамках библиотеки STL.

В разделе 7.1.2 перечислены все общие контейнерные операции и отмечены те из них, которые требуются для контейнеров STL. Тем не менее это не значит, что соответствовать принципам библиотеки STL можно, лишь удовлетворив *всем* этим требованиям. Как правило, достаточно лишь частично выполнить эти требования. Даже некоторые стандартные контейнеры STL нарушают эти требования. Например, класс `forward_list` не имеет функции-члена `size()`, а класс `array` не соответствует общему требованию, что контейнер STL, инициализированный конструктором по умолчанию, должен быть пустым.

6.13.2. Наследование типов библиотеки STL

Другой вопрос — можно ли расширить возможности типов из библиотеки STL, создав производные классы и добавив в них функциональность. Обычно это невозможно. По причинам, связанным с производительностью, все классы библиотеки STL не содержат виртуальных функций, и поэтому не поддерживают полиморфизм посредством открытого наследования. Для того чтобы добавить в контейнер новую функциональность, необходимо определить новый класс, который внутренне использует классы библиотеки STL, или применить закрытое наследование.